

EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 4-BN: Binary Ninja Personal — Resolve, Hack, and Patch the RP2350 (Raw `.bin`)

ARM Cortex-M33 • Binary Ninja • OpenOCD

October 3, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build the two lesson projects with `Release` and get both an `.elf` and a raw `.bin`
 - Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
 - Load the raw `.bin` into Binary Ninja at `0x10000000`
 - **Break at `main`** on live silicon, even though `main` can move between programs
 - **Hack a running target live** by editing a register in Binary Ninja's Registers widget
 - **Resolve the functions in the Binary Ninja GUI** using the ELF symbol map
 - **Patch** the bytes that control the behavior, export the image, and flash it
-

How This Guide Works

The build produces two files for each project:

File	What it is	How we use it
<code>.elf</code>	The linked image with a full symbol table	Ground truth for every function address and name
<code>.bin</code>	The raw flash image, no headers, no symbols	The image we load into Binary Ninja and reverse

The `.bin` is built **from** the `.elf`, so the ELF tells you exactly what is at every address. We use the ELF symbol map to resolve functions in Binary Ninja, and we reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

Build `Release`, **not** `Debug`. Every address in this guide matches the Week 4 lesson, and the Week 4 lesson is a `Release` build. `Release` optimizes the code the same way the original lesson was built: it folds `age = 42` away in Project 1 and inlines `blink_and_print` into `main` in Project 2. If you build `Debug`, the SDK function addresses move and Project 2 keeps a separate `blink_and_print`, so nothing lines up. Always build `Release` for this lesson.

The order is **dynamic first, static second**, twice — once per project:

1. Break on the live target and prove what the code does.
2. Hack it live in the debugger and watch the behavior change.
3. Resolve the functions in Binary Ninja using the ELF symbol map.
4. Patch the bytes, export, convert, and flash.

Project	Prints	Also does	The hack
0x0005_intro-to-variables	age: 43	loops on <code>printf</code>	change 43 to 70
0x0008_uninitialized-variables	age: 0	blinks the red LED on GPIO 16	change 0 to 66, move the LED to GPIO 17

Addresses come from your build. Every address here is from the `Release` build produced in Step 3. Confirm against your own `.elf` with the command in Step 4.

Part 1: Build, Flash, and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **Binary Ninja Personal** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

```
brew install cmake ninja
```

- Install **Binary Ninja Personal** and complete its license activation.
- Install the **Arm GNU Toolchain**, or let the VS Code Pico extension manage it.

Linux x64

```
sudo apt install cmake ninja-build gcc-arm-none-eabi libnewlib-arm-none-eabi git python3 openocd minicom
```

- Install **Binary Ninja Personal** and complete its license activation.

Step 2: Verify your tools are the right architecture (do not skip this)

On **macOS Apple Silicon**, the most common failure is an Intel `x86_64` tool on your `PATH`:

```
zsh: bad CPU type in executable: cmake
```

You may have **two Homebrews**: the arm64 one at `/opt/homebrew` and the Intel one at `/usr/local`. If `/usr/local/bin` wins, every `brew` tool is x86_64. Check:

```
file "$(which cmake)"
file "$(which ninja)"
file "$(which arm-none-eabi-gdb)"
file "$(which arm-none-eabi-nm)"
file "$(which openocd)"
file "$(which telnet)"
```

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

To make it permanent, add that `export` to `~/.zshrc`. Do not use Rosetta as a fix; OpenOCD and GDB are exactly the kind of programs where a translation layer produces failures that look like debugger bugs.

telnet is special. macOS no longer ships `telnet`, and the Homebrew build is often the Intel one, so `telnet 127.0.0.1 4444` fails with `bad CPU type in executable`. Your `brew` command itself may also be the Intel one: if `brew install telnet` fails with `.../portable-ruby/.../ruby: Bad CPU type in executable`, you are running the Intel Homebrew. Call the Apple Silicon Homebrew explicitly:

```
/opt/homebrew/bin/brew install telnet
```

If you would rather not install anything, macOS ships an arm64 `nc`, which can connect to the same OpenOCD port:

```
nc 127.0.0.1 4444
```

Windows x64 and **Linux x64** do not have this problem. Skip to Step 3.

Step 3: Build the two projects with `Release`

Run this once inside `0x0005_intro-to-variables/` and once inside `0x0008_uninitialized-variables/`:

```
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Each build directory now contains the pair we need:

- `0x0005_intro-to-variables/build/0x0005_intro-to-variables.elf` and `.bin` — `.bin` is **15292** bytes
- `0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.elf` and `.bin` — `.bin` is **15668** bytes

If the ARM toolchain is not on your `PATH`, add `-DPICO_TOOLCHAIN_PATH=...`:

OS	Typical toolchain path
Windows x64	<code>C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2.rel1/bin</code>
macOS Apple Silicon	<code>/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin</code>

OS	Typical toolchain path
Linux x64	/usr

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on each ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x0005_intro-to-variables.elf | grep -E ' [Tt] '
arm-none-eabi-nm -n --defined-only build/0x0008_uninitialized-variables.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x0005_intro-to-variables.elf | Select-String ' [Tt] '
```

Each line is `address type name`. The `T / t` type is a function. Here are the functions this lesson uses.

Project 1 — 0x0005_intro-to-variables :

Address	ELF symbol	Role
0x1000015c	<code>_reset_handler</code>	reset entry
0x10000186	<code>platform_entry</code>	calls <code>runtime_init</code> , <code>main</code> , <code>exit</code>
0x1000019a	<code>data_cpy</code>	copies <code>.data</code> from flash to SRAM
0x100001e4	<code>_init</code>	runs <code>.init_array</code>
0x10000210	<code>frame_dummy</code>	C runtime boilerplate
0x10000234	<code>main</code>	the lesson function
0x10000248	<code>gpio_set_function</code>	SDK GPIO helper
0x10002cfc	<code>exit</code>	C runtime exit
0x10002d04	<code>runtime_init</code>	SDK runtime init
0x10002f54	<code>stdio_init_all</code>	SDK serial init
0x100030e4	<code>__wrap_printf</code>	the <code>printf</code> wrapper

Project 2 — 0x0008_uninitialized-variables :

Address	ELF symbol	Role
0x1000015c	<code>_reset_handler</code>	reset entry
0x10000186	<code>platform_entry</code>	calls <code>runtime_init</code> , <code>main</code> , <code>exit</code>
0x1000019a	<code>data_cpy</code>	copies <code>.data</code> from flash to SRAM

Address	ELF symbol	Role
0x100001e4	_init	runs .init_array
0x10000210	frame_dummy	C runtime boilerplate
0x10000234	main	the lesson function (blink_and_print inlined)
0x10000278	gpio_set_function	SDK GPIO helper
0x100002b4	gpio_init	SDK GPIO init
0x10000d10	sleep_ms	SDK delay
0x10002e74	exit	C runtime exit
0x10002e7c	runtime_init	SDK runtime init
0x100030cc	stdio_init_all	SDK serial init
0x1000325c	__wrap_printf	the printf wrapper

`main` is `0x10000234` in both projects. In Project 2 the static void `blink_and_print` helper is inlined into `main` by the Release optimizer, so it does not appear as a separate symbol. That is why both projects put `main` at the same address. In a Debug build it stays separate and `main` moves — another reason to build Release.

Step 5: Flash Project 1 and confirm age: 43

A `.bin` has no headers, so OpenOCD must be told the base address `0x10000000`. From the repository root:

macOS Apple Silicon / Linux x64:

```
./flash.sh 0x0005_intro-to-variables/build/0x0005_intro-to-variables.bin
```

Windows x64 (PowerShell):

```
.\flash.ps1 -Bin 0x0005_intro-to-variables\build\0x0005_intro-to-variables.bin
```

Wait for wrote 15292 bytes ... and **** Verified OK ****. Open a serial monitor at **115200** baud:

- **Windows x64:** PuTTY -> Connection type **Serial**, the Pico's COM port, speed `115200`.
- **macOS Apple Silicon:** `screen /dev/tty.usbmodem* 115200` (quit with `Ctrl-A` then `K`).
- **Linux x64:** `minicom -D /dev/ttyACM0 -b 115200`.

```
age: 43
age: 43
age: 43
...
```

Step 6: Flash Project 2 and confirm age: 0 + red LED

```
./flash.sh 0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.bin
```

Wait for `wrote 15668 bytes ...`. The serial monitor shows:

```
age: 0
age: 0
age: 0
...
```

and the **red LED on GPIO 16** blinks once per second.

Part 2: Load the Raw `.bin` into Binary Ninja

Start from a fresh Binary Ninja state. If you already have a `.bndb` for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 7: Bring the raw `.bin` into Binary Ninja

A raw `.bin` has no headers, so Binary Ninja cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, Binary Ninja may load it at address `0x0` with a guessed architecture, and every address in this lesson will be wrong.

1. Choose `File -> Open with Options...` (do **not** use plain `File -> Open`).
2. Select `0x0005_intro-to-variables/build/0x0005_intro-to-variables.bin`.
3. In the loader options, set:
 - **Architecture:** `thumb2` (the ARMv7-M / ARMv8-M Thumb-2 architecture, which covers the Cortex-M33)
 - **Platform:** `thumb2`
 - **Base Address:** `0x10000000` (the XIP flash base)
4. Click **Open**.

Binary Ninja analyzes the image and opens the linear view.

Verify the load before going further. Press `G`, type `0x10000000`, and read the first two words:

```
0x10000000  0x20082000  initial stack pointer
0x10000004  0x1000015d  reset vector (bit 0 = Thumb)
```

If you instead see data at `0x00000000`, or a vector word without bit 0 set, close the tab and repeat with `Open with Options`. The Cortex-M33 only executes Thumb-2, so `thumb2` is the only correct architecture.

Console equivalent:

```
load("0x0005_intro-to-variables/build/0x0005_intro-to-variables.bin",
     options={"loader.imageBase": 0x10000000, "loader.platform": "thumb2"})
```

Step 8: Save it as a Binary Ninja database (.bndb)

Binary Ninja never writes back into the `.bin`. Your names, comments, types, and patches live in a separate `.bndb` database. Save one now, before you make any changes:

1. Choose `File -> Save As...`
2. Save it next to the image as `0x0005_intro-to-variables.bndb`.
3. From now on, save with `File -> Save` (`Cmd+S` on macOS, `Ctrl+S` on Windows/Linux) whenever you rename or patch.

The two files have different roles:

File	Role
<code>0x0005_intro-to-variables.bin</code>	the raw firmware image; Binary Ninja never modifies it
<code>0x0005_intro-to-variables.bndb</code>	your analysis database: names, types, comments, and patches

When you come back later, **open the .bndb**, not the `.bin`; that restores all your work. If a database gets messy, delete the `.bndb` and re-import the `.bin` from Step 7 — the firmware is never at risk. You export the patched image out of this view later, in Step 19.

Step 9: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment. Breakpoints are set from the OpenOCD command port, not from the GUI — see Step 13.

macOS function keys: the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

Part 3: Dynamic — Break at `main` and Hack Live (Project 1)

Step 10: Start OpenOCD as a live debug server

Make sure no other OpenOCD is running; a forgotten server holds port `3333`.

macOS / Linux:

```
ps aux | grep -i openocd
```

Windows (PowerShell):

```
Get-Process | Where-Object { $_.ProcessName -like '*openocd*' }
```

Stop any leftover server gracefully:


```
pkill -TERM -f openocd
```

Start the server **parked at** `main`:

macOS Apple Silicon / Linux x64:

```
BP_ADDR=0x10000234 ./debug-server.sh
```

Windows x64 (PowerShell):

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1
```

Wait for:

```
Info : [rp2350.dap.core0] Examination succeed
Startup breakpoint at 0x10000234 (2-byte hardware execute, one-shot).
Info : starting gdb server for rp2350.dap.core0 on 3333
Info : Listening on port 3333 for gdb connections
```

`BP_ADDR` **parks the core at** `main` **before any client connects**. The script arms a 2-byte hardware breakpoint and then does the startup `reset run`, so the core runs from the vector table and stops at your address with no debugger attached yet. When Binary Ninja connects a moment later, the first thing it reads is already the truth: `Stopped at 0x10000234`. This is the whole reason the lab works cleanly — you never have to drive a reset from outside the GUI.

Use the address you actually want to stop at:

What you want to stop at	Project 1 0x0005	Project 2 0x0008	Command
<code>main</code> (once per reset)	<code>0x10000234</code>	<code>0x10000234</code>	<code>BP_ADDR=0x10000234 ./debug-server.sh</code>
The loop — the <code>printf</code> call, hit every iteration	<code>0x1000023e</code>	<code>0x1000024e</code>	<code>BP_ADDR=0x1000023e ./debug-server.sh</code>

```
BP_ADDR=0x10000234 ./debug-server.sh # park at main
BP_ADDR=0x1000023e ./debug-server.sh # park in the loop instead
```

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1 # park at main
$env:BP_ADDR="0x1000023e"; .\debug-server.ps1 # park in the loop
```

Note the loop address is not the same in both projects. Project 2 does more setup before the loop, so its `bl __wrap_printf` sits at `0x1000024e`, not `0x1000023e`. Both were verified against the Release `.elf` with `arm-none-eabi-objdump` and confirmed live on hardware.

This startup stop is single-use. OpenOCD flushes breakpoints when a client connects, so this one is gone once Binary Ninja attaches — fine for `main`, which only runs once per reset. Every breakpoint after that is set from the Binary Ninja GUI (Step 13) and is repeatable. To stop at `main` again, restart the server with `BP_ADDR` and reconnect.

Exactly one core. The line must say `core0` and must **not** mention `core1`. `Core1` is never started by this firmware; exposing it makes Binary Ninja read `core1`'s reset-state registers, which are not real addresses, and OpenOCD floods the log with `Failed to read memory at 0xf0000000`. The scripts already use `USE_CORE=0`; do not change it.

Windows driver note: the Debug Probe must use the **WinUSB** driver. If OpenOCD reports `unable to open CMSIS-DAP device`, install it with [Zadig](#) (select `Debug Probe (CMSIS-DAP)` -> `WinUSB`).

Step 11: Connect Binary Ninja to the GDB server

1. Make sure the image is open and analyzed (Part 2) and the server from Step 10 is running (parked at `main`).
2. Choose `Debugger -> Connect to Remote Process`.
3. In the **adapter** dropdown, select **GDB MI**.
4. In the **connect** settings group, set **IP Address** to `127.0.0.1` and **Port** to `3333`.
5. Set **Full GDB Executable Path** to your `arm-none-eabi-gdb`. On macOS the build that works is the **14.2.rel1** toolchain:

```
/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin/arm-none-eabi-gdb
```

6. Click **Accept**.

Use the GDB MI adapter. It launches a real `arm-none-eabi-gdb --interpreter=mi2` and lets Binary Ninja drive it, so breakpoints and stepping go through real GDB — which sends the correct 2-byte breakpoint length and handles step-over itself. Verified working end to end: connect, GUI breakpoints (`F2` / **Add Hardware Breakpoint...**), **Step Into** / **Step Over**, and register edits. Stops are reported as `Breakpoint` (not `SingleStep`).

Do NOT have any breakpoints set in Binary Ninja before you connect. With the GDB MI adapter, attaching while Binary Ninja already has a breakpoint **hangs the session**. Start the server parked with `BP_ADDR` (Step 10), connect, and only add hardware breakpoints *after* the connection is up. This is a Binary Ninja bug; it is the single most common GDB MI failure.

The GDB executable path matters. Use the **14.2.rel1** build above. The `13.3.rel1` build — which is what `/opt/homebrew/bin/arm-none-eabi-gdb` symlinks to — did **not** connect in testing.

Do not pick Corellium. Binary Ninja's adapter dropdown also lists **Corellium**, which is for Corellium's virtual devices and expects an API token, not a local OpenOCD server. It is not the adapter for this lab. The dropdown is a combo box, so an accidental arrow-key press can land on it — always read the label back and confirm it says **GDB MI** before clicking **Accept**.

The adapter and port are not saved in the .bndb. Every time you relaunch Binary Ninja you must re-select **GDB MI**, re-enter port `3333`, and re-set the GDB path.

Watch for an off-screen error dialog. When a connection fails, Binary Ninja pops a `Binary Ninja critical alert` window that can be positioned mostly outside the main window (seen at `2430,331` with the main window at `2560,30`), which makes it look like nothing happened. If the connect seems to do nothing, check your other display.

The target keeps running. Open the **Registers** tab (bug icon) and confirm you see live values. `pc` inside `0x10003xxx` and `sp` just below `0x20082000` are healthy.

If `pc` is `0x00000088`, `0x000000ec`, or `sp` is `0xf0000000`, the session is bad. Restart the server, then restart Binary Ninja (a server restart while attached leaves Binary Ninja in a stale session), and connect again.

Step 12: Find `main` without relying on its address

`main` can move between programs, so we do not guess it. We follow the one fixed path to it. Press `G` and go to `0x10000000`:

```
0x10000000  0x20082000  initial stack pointer (top of SRAM)
0x10000004  0x1000015d  reset vector
```

Bit 0 of a vector is the Thumb bit, so `0x1000015d` means "start at `0x1000015c`". That is `_reset_handler`. Follow the reset path to `0x10000186`, `platform_entry`:

```
10000186 <platform_entry>:
10000186: 4914  ldr r1, [pc, #80] ; @ 0x100001d8
10000188: 4788  blx r1             ; runtime_init
1000018a: 4914  ldr r1, [pc, #80] ; @ 0x100001dc
1000018c: 4788  blx r1             ; main <-- the fixed anchor
1000018e: 4914  ldr r1, [pc, #80] ; @ 0x100001e0
10000190: 4788  blx r1             ; exit
10000192: be00  bkpt 0x0000
```

The middle `blx` at `0x1000018c` is the call to `main`. `platform_entry` is byte-identical in both projects, so `0x1000018c` catches `main` no matter where the linker placed it. The literal pool at `0x100001dc` holds `main | 1`, and clearing bit 0 gives `0x10000234`.

Step 13: Set a hardware breakpoint (from OpenOCD, because the GUI cannot)

First: decide where you want to stop

There are two different jobs, and they use **different addresses and different methods**. Mixing them up is the most common source of confusion in this lab.

You want to stop at	Address	How	Repeatable?
<code>main</code>	<code>0x10000234</code>	Start the server with <code>BP_ADDR=0x10000234 ./debug-server.sh</code> , then connect Binary Ninja. No <code>nc</code> , no <code>bp</code> .	No — <code>main</code> runs once per reset, so this is a one-shot first stop.
The loop (printf call)	Project 1 <code>0x1000023e</code> , Project 2 <code>0x1000024e</code>	Connect Binary Ninja first , then <code>nc 127.0.0.1 4444</code> and <code>bp <addr> 2 hw</code> , then click Resume .	Yes — fires on every iteration.

For `main`, use `BP_ADDR`. It is one command and Binary Ninja shows `Stopped at 0x10000234` immediately when it connects:

```
BP_ADDR=0x10000234 ./debug-server.sh
```

Then `Debugger -> Connect to Remote Process -> GDB RSP -> Accept`. That is the whole procedure for `main`.

There is a trap if you instead arm `main` over the command port while Binary Ninja is already connected. It does stop the core at `0x10000234` (verified: `pc=0x10000234`), but Binary Ninja's status bar keeps showing its *previous* stop location, e.g. `Stopped (InitialBreakpoint) at 0x1000320c`, because it never saw a stop event for `main`. The core and the display disagree. To resync Binary Ninja you must **Detach and**

reconnect (verified: the status then reads `Stopped (InitialBreakpoint) at 0x10000234`). `BP_ADDR` avoids this entirely because Binary Ninja connects while the core is already parked at `main`, so the first thing it reads is the truth.

Everything below is the loop workflow, which is what you want for stepping and for the live `r1` hack.

Known Binary Ninja bug (2026-10-02, BN 6.0.10601): you cannot set a breakpoint from the GUI on this target. Binary Ninja sends the RSP packet `Z0,10000234,1` — a **1-byte** breakpoint. The Cortex-M33 FPB comparators are halfword-based, so OpenOCD rejects it:

```
Info : cortex_m.c:1908 cortex_m_add_breakpoint(): [rp2350.dap.core0] only breakpoints of two
bytes length supported
Error: breakpoints.c:86 breakpoint_add_internal(): [rp2350.dap.core0] can't add breakpoint:
resource not available
```

This affects **every** address and **both** methods below — `Debugger -> Toggle Breakpoint (F2)` and `Debugger -> Add Hardware Breakpoint...` (`F3`) with **Type** `Hardware Execute`. The dialog's **Size** field is disabled and hard-coded to `1`, so there is no UI escape hatch, and `gdb_breakpoint_override` hard changes nothing (both the `Z0` and `Z1` paths end in the same rejected call). Plain GDB works because `hbreak` sends a length of `2`.

Do not fight the dialog. Arm the breakpoint from the OpenOCD command port instead. The order matters — see the warning below.

1. Leave Binary Ninja connected (Step 11). Do **not** detach.
2. Open the OpenOCD command port in a second terminal:

```
nc 127.0.0.1 4444      # or: telnet 127.0.0.1 4444
```

3. Arm a 2-byte **execute** breakpoint at `main`:

```
bp 0x10000234 2 hw
```

`bp <addr> 2 hw` sets an execute-type hardware comparator.

4. **Do not run** `reset run` **from the port while Binary Ninja is connected.** `main` runs once per reset, so a breakpoint on `main` only fires if the core resets *after* it is armed — but a reset driven from the command port makes Binary Ninja miss the stop event. The target halts at `0x10000234` while Binary Ninja's view keeps showing wherever it last stopped, so the Step/Resume buttons act on the wrong address. This is the single most common "it worked for a second and then stopped" symptom. The reliable way to stop at `main` is to arm it **before** Binary Ninja connects, with `BP_ADDR=0x10000234` (Step 10): Binary Ninja's first read is then already the truth. If you did reset while connected, **Detach and reconnect** to resync (verified: the status line then reads `Stopped at 0x10000234`).

Arm the breakpoint only *after* Binary Ninja is connected. Any breakpoint set before a client attaches is destroyed the moment that client connects. OpenOCD logs it explicitly:

```
Info : accepting 'gdb' connection on tcp/3333
Debug: breakpoints.c:328 breakpoint_remove_all_internal(): [rp2350.dap.core0] Delete all breakpoints
```

This affects every arrangement:

- **hbreak in GDB, then detach** — detaching zeroes the comparators; `mdw 0xE0002000` reads back all zeros.
- **Arming over telnet before Binary Ninja connects** — the connect flushes it.
- **BP_ADDR on the startup command line (Step 10)** — it does fire and does park the core at your address, but it is flushed on connect, so it is a single-use first stop. Verified by reading the comparators: `0x10000235` before attach, all zeros after.

Attach first, then arm. Verified working at `0x10000234` and again at `0x1000023e`.

What "attach first, then arm" actually means

Two different channels are in play, and they are easy to confuse:

Channel	Port	What it is	How you use it
GDB server	3333	What Binary Ninja talks to	You never type in this one. BN connects to it via the GUI.
Telnet command port	4444	A plain text prompt for driving OpenOCD by hand	You type commands here, at an <code>OpenOCD></code> prompt.

So the sequence is literally:

1. **Terminal 1** — run `./debug-server.sh` and leave it running.
2. **Binary Ninja GUI** — the menu bar has **Debugger -> Connect to Remote Process**. Pick **GDB RSP** in the adapter dropdown, then **Accept**. (This is the "attach" step. It is a GUI menu item, not something you type at a prompt.)
3. **Terminal 2** — open the command port and get a prompt:

```
nc 127.0.0.1 4444
```

You should see an `OpenOCD>` prompt.

4. **At that prompt**, type this one line and press Enter:

```
bp 0x1000023e 2 hw
```

Expect breakpoint set at `0x1000023e`. Then click **Resume** in Binary Ninja and the loop breakpoint fires on the next iteration.

What the command means, and what `rbp` is for:

Command	Full name	What it does
<code>bp <addr> 2 hw</code>	breakpoint	Arms a hardware breakpoint at that address. The <code>2</code> is the instruction length in bytes, and <code>hw</code> means hardware rather than software.
<code>rbp <addr></code>	remove breakpoint	Deletes the breakpoint at that address. Address only — no <code>2</code> , no <code>hw</code> .
<code>rbp all</code>	remove all	Deletes every breakpoint.

The `2` is not optional decoration. Binary Ninja sends `1`, and Cortex-M rejects that with `only breakpoints of two bytes length supported`, which is the whole reason this lab arms breakpoints here instead of using the GUI.

Use `0x1000024e` instead of `0x1000023e` on Project 2.

`no breakpoint at address ... found` **is not a problem**. You will only need `rbp` to *move* a breakpoint you set earlier. On a fresh run there is nothing to remove, and `rbp` answers with:

```
[rp2350.dap.core0] no breakpoint at address 0x10000234 found
Error during removal of breakpoint at address 0x10000234
```

That is OpenOCD saying "there was nothing there", not a failure. Ignore it and carry on with the `bp` line. Confirmed live: after that error, `bp 0x1000023e 2 hw` armed cleanly and the comparator read `0x1000023f`.

5. When you are done, `quit` at the `OpenOCD>` prompt. Closing the prompt does not kill the server.

To confirm the breakpoint is really armed, run `mdw 0xE0002000 4` at the prompt. You want `0x1000023f` in the third word — that is `0x1000023e | 1`, where the low bit marks the address as Thumb. An all-zero result means it got wiped, which means you armed it before Binary Ninja connected.

Verified live: comparator read `1000023f` after arming, `00000000` the moment Binary Ninja connected (proving the wipe), then `1000023f` again after re-arming over the prompt. **Resume** landed at `pc=0x1000023e`, `r1=0x2b` and re-caught on every subsequent **Resume**.

What you will and will not see. Binary Ninja labels these stops `SingleStep` rather than `Breakpoint`, because it has no idea a breakpoint exists, and the **Breakpoints** widget stays empty. That is expected and harmless — the core really is halted on a hardware comparator you armed. To confirm what is armed, read the FPB comparator registers on the command port: each armed breakpoint appears at `0xE0002008 + 4n` as `<address | 1>`.

Never use Binary Ninja's Restart button. On RP2350 it resets and halts inside the boot ROM (`pc=0x88`, `sp=0xf0000000`). To reset cleanly, use `BP_ADDR` on a fresh server start, or **Detach**, send `reset run` from the command port, and reconnect — never `reset run` while attached (it desyncs Binary Ninja's view; see Step 13).

You often do not need a reset. `main` is an infinite loop, so its body from `0x1000023a` to `0x10000242` runs forever. Arm a breakpoint inside that loop, such as the `printf` call at `0x1000023e`, then click **Resume** in Binary Ninja — it fires on the next iteration with no reset at all. Step 14 uses exactly that.

Stepping: two bugs that stop it working, and the fixes

If **Step Into** / **Step Over** in Binary Ninja do nothing — the PC stays exactly where it is, no matter how many times you click — there are two independent causes, both confirmed on this setup by reading the OpenOCD GDB log (`log_output <file> + debug_level 3`).

Cause 1: the hwthread RTOS makes OpenOCD fake the step. `target/rp2350.cfg` creates `core0` with `-rtos hwthread`, which registers a fake RTOS whose current thread is `coreid + 1 = 1`. Binary Ninja single-steps with the packet `vCont;s` and no thread id, i.e. thread 0. OpenOCD's `gdb_server.c` sees `rtos->current_thread (1) != thread_id (0)` and takes its "fake step" path, sending a stop reply **without ever stepping the core**:

```
Debug: gdb_server.c:3094 gdb_handle_vcont_packet(): target rp2350.dap.core0 single-step thread 0
Debug: gdb_server.c:3112 gdb_handle_vcont_packet(): fake step thread 0
Debug: gdb_server.c:397  gdb_log_outgoing_packet(): sending packet: $T05thread:0000000000000000;#a6
```

The fix is to drop the RTOS. `debug-server.sh` and `debug-server.ps1` now pass this automatically, right after the target config is read:

```
rp2350.dap.core0 configure -rtos none
```

If you start OpenOCD by hand or with an older copy of the script, add that line. With the RTOS gone, `vCont;s` reaches `cortex_m_step()` and the core really moves.

Cause 2: a breakpoint sitting on the current PC blocks stepping. Binary Ninja's step is passed to OpenOCD as a step *over a breakpoint* (`target_step(..., current_pc=1, ...)`). When a breakpoint is already armed at the address you are halted on, OpenOCD tries to add its own breakpoint at that same address and fails:

```
Error: breakpoints.c:56 breakpoint_add_internal(): [rp2350.dap.core0] Duplicate Breakpoint address: 0x1000023e (BP 9)
Debug: cortex_m.c:884 cortex_m_debug_entry(): entered debug state ... at PC 0x1000023e
```

The core steps and immediately re-traps on the same comparator, so the PC appears not to move. The fix is to **remove the breakpoint before you step**:

```
rbp 0x1000023e
```

Then click **Step Into** or **Step Over**; the PC advances normally. Verified live: after `rbp 0x1000023e`, Step Into walked `0x1000023e -> 0x100030e4 -> 0x100030e6 -> 0x100030e8 -> ...`. This is why Step 14 below removes the breakpoint before stepping over the `printf` call.

Note: both causes look identical from the GUI — a click that does nothing. The PC never moving, with no error dialog, is the signature. Check the OpenOCD log for `fake step` (cause 1) or `Duplicate Breakpoint` (cause 2) to tell them apart.

Step 14: HACK IT LIVE — change the printed value

`main` loads the constant `0x2b` (43) into `r1` and calls `printf` on every iteration. We break on that call in the GUI and change it live.

1. Press **G**, go to `0x1000023e` (the `bl __wrap_printf`).
2. In the terminal, connect to the OpenOCD command port if you are not already there:

```
nc 127.0.0.1 4444
```

Then type this line at the `OpenOCD>` prompt and press Enter:

```
bp 0x1000023e 2 hw
```

Expect breakpoint set at `0x1000023e`. If you previously armed a breakpoint elsewhere, clear it first with `rbp <that address>` (`rbp` all clears every one). `rbp` means *remove breakpoint* and takes an address only; running it when nothing is armed prints `no breakpoint at address ... found`, which is harmless.

3. Click **Resume** in Binary Ninja. The target is already running the loop, so the comparator fires on the next iteration. Binary Ninja stops with the program counter at `0x1000023e` and `r1 = 0x2b`.
4. Open the **Registers** widget (bug icon -> **Registers**).
5. Find `r1`. Its value is `0x2b`.
6. **Double-click the value, type 46, and press Enter.** Binary Ninja parses the new value as hex, so `46` means `0x46` (70). The edited value turns **orange**.
7. **Move the breakpoint past the call, then Resume.** You want `printf` to run once and then stop, so put the breakpoint on the instruction *after* the call. At the `OpenOCD>` prompt:

```
rbp 0x1000023e
bp 0x10000242 2 hw
```

`0x10000242` is the `b.n` that closes the loop. Two reasons not to just click **Step Over** here: a breakpoint left on the current PC re-traps the step (see the stepping note in Step 13), and Binary Ninja's **Step Over** steps *into* `__wrap_printf` on this raw `.bin` because the image carries no symbol for the call. Moving the breakpoint to the return site is deterministic.

8. Click **Resume** in Binary Ninja. The core executes `bl __wrap_printf` with `r1 = 0x46`, so this iteration prints `age: 70`, then stops at `0x10000242`.
9. Look at your serial monitor — the `screen` session on the Pico's USB serial port — and at the **Target** tab in Binary Ninja:

```
age: 70
```

You changed a running program's output without touching the binary.

Step 14b: HACK THE STRING LIVE — change age: to foo:

The text "age: %d\r\n" lives in flash (.rodata) at 0x100034a0, and flash is **read-only at runtime** — a debugger write there does not stick (verified: writing 0x66 to 0x100034a0 read back unchanged). So you cannot overwrite the text in place. Instead you redirect the pointer: at the printf call, r0 holds the string address, so you point r0 at a replacement string you place in RAM.

1. Arm the breakpoint at the printf call and hit it, exactly as in Step 14 steps 1-3. At the stop, r0 = 0x100034a0 and r1 = 0x2b.
2. Put the replacement string into free RAM at 0x20080000. Binary Ninja has no memory editor, so this one step uses the command port. At the OpenOCD> prompt:

```
mww 0x20080000 0x3a6f6f66
mww 0x20080004 0x0d642520
mww 0x20080008 0x0000000a
```

That writes the bytes 66 6f 6f 3a 20 25 64 0d 0a 00 = "foo: %d\r\n\0" (three little-endian words).

3. In the **Registers** widget, double-click r0 and set it to 0x20080000. It turns orange.
4. Move the breakpoint past the call and Resume:

```
rbp 0x1000023e
bp 0x10000242 2 hw
```

The core runs printf with r0 pointing at your RAM string and r1 = 0x2b, so this iteration prints:

```
foo: 43
```

then stops at 0x10000242.

Like the value hack, this is **one iteration only**: the loop reloads r0 (and r1) from flash on every pass, so the next line is age: 43 again. The permanent version is the static patch in Step 19b.

Step 15: Why the hack reverts (and why we patch next)

Press **Resume**. The loop branches back to 0x1000023a, which reloads movs r1, #43, so the next line is age: 43. The live edit changed one iteration only. There is no variable in memory to change; the value is baked into the instruction. To make age: 70 permanent we must patch the instruction. That is the static pass.

Press **Pause** to stop the output flood.

Part 4: Static — Resolve the Functions in Binary Ninja and Patch (Project 1)

Step 16: Resolve the functions in the Binary Ninja GUI

Now we use the ELF symbol map from Step 4 to name the functions in Binary Ninja. For each row below:

1. Press **G** and type the address.
2. Press **N** and type the ELF symbol name.
3. For functions with arguments, press **Y** and set the signature shown.

This is **our code plus the library functions it actually calls** — not the whole SDK. `main` only calls `stdio_init_all` and `printf`, so we follow that chain down: `stdio_init_all` pulls in the stdio/UART setup, and `printf` lands in the SDK's `__wrap_printf`.

The call chain for this project:

```
main
├── stdio_init_all
│   ├── stdio_uart_init — gpio_set_function, uart_init
│   ├── stdio_set_driver_enabled
│   ├── stdio_out_chars_crlf
│   ├── stdio_put_string — strlen
│   ├── time_us_64
│   └── __wrap_printf — __wrap_vprintf
```

Project 1 — resolve every function in that chain:

Address	Rename to (N)	Signature (Y)
0x1000015c	<code>_reset_handler</code>	—
0x10000186	<code>platform_entry</code>	<code>void platform_entry(void)</code>
0x1000019a	<code>data_cpy</code>	<code>void data_cpy(void*, void*, void*)</code>
0x100001e4	<code>_init</code>	<code>void _init(void)</code>
0x10000210	<code>frame_dummy</code>	<code>void frame_dummy(void)</code>
0x10000234	<code>main</code>	<code>int main(void)</code>
0x10002cfc	<code>exit</code>	<code>void exit(int)</code>
0x10002d04	<code>runtime_init</code>	<code>void runtime_init(void)</code>
0x10002f54	<code>stdio_init_all</code>	<code>bool stdio_init_all(void)</code>
0x100032a0	<code>stdio_uart_init</code>	<code>void stdio_uart_init(void)</code>
0x10002f2c	<code>stdio_set_driver_enabled</code>	<code>void stdio_set_driver_enabled(stdio_driver_t*, bool)</code>
0x10002d30	<code>stdio_out_chars_crlf</code>	<code>void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)</code>
0x10002e40	<code>stdio_put_string</code>	<code>int stdio_put_string(const char*, int, bool, bool)</code>
0x100030e4	<code>__wrap_printf</code>	<code>int __wrap_printf(const char*, ...)</code>
0x10003020	<code>__wrap_vprintf</code>	<code>int __wrap_vprintf(const char*, va_list)</code>
0x10000248	<code>gpio_set_function</code>	<code>void gpio_set_function(uint, gpio_function_t)</code>
0x10000e10	<code>uart_init</code>	<code>uint uart_init(uart_inst_t*, uint)</code>
0x10000da0	<code>time_us_64</code>	<code>uint64_t time_us_64(void)</code>
0x100033e0	<code>strlen</code>	<code>size_t strlen(const char*)</code>

`__wrap_printf` **is the real symbol**. `printf` in our source compiles to the SDK's `__wrap_printf` (which forwards to `__wrap_vprintf`). Rename it `printf` if you prefer the lesson's shorthand, but `__wrap_printf` is what the ELF says.

`stdio_init_all` **returns** `bool`, **not** `void` — `_Bool stdio_init_all(void)` in the ELF. The `main` source ignores the return value, so the decompiler still reads fine.

Shortcut: instead of renaming by hand, paste this into Binary Ninja's Python console (`Plugins -> Python Console`). It applies the same symbol map programmatically:

```
from binaryninja import Symbol, SymbolType
symbols = {
    0x1000015c: "_reset_handler",      0x10000186: "platform_entry",
    0x1000019a: "data_cpy",           0x100001e4: "_init",
    0x10000210: "frame_dummy",        0x10000234: "main",
    0x10002cfc: "exit",               0x10002d04: "runtime_init",
    0x10002f54: "stdio_init_all",     0x100032a0: "stdio_uart_init",
    0x10002f2c: "stdio_set_driver_enabled",
    0x10002d30: "stdio_out_chars_crlf", 0x10002e40: "stdio_put_string",
    0x100030e4: "__wrap_printf",      0x10003020: "__wrap_vprintf",
    0x10000248: "gpio_set_function",   0x10000e10: "uart_init",
    0x10000da0: "time_us_64",         0x100033e0: "strlen",
}
for addr, name in symbols.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
```

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main`. It reads:

```
int32_t main(void)
{
    stdio_init_all();
    do
    {
        printf("age: %d\r\n", 0x2b);
    } while (true);
}
```

The `0x2b` is the value we edited live. Now make it permanent.

Step 18: Patch `0x2b` to `0x46` in the GUI

Go to `0x1000023a`:

```
1000023a: 212b    movs   r1, #43 ; 0x2b
```

The halfword is `0x212b`, stored little-endian as `2b 21`. The immediate is the low byte, so the byte at the instruction's own address is `0x2b`. Change it to `0x46` (70).

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Toggle the lock off so editing is enabled.
3. Go to `0x1000023a` and change the byte `2B` to `46`.
4. Return to the linear view, right-click the function -> `Reanalyze`.

Option B — Python console:

```
bv.write(0x1000023a, b"\x46")
print(hex(bv.read(0x1000023a, 1)[0])) # -> 0x46
```

After reanalysis the instruction reads `movs r1, #70`.

Step 18b: Patch the string `age: to foo: in the GUI`

The format string `"age: %d\r\n"` starts at `0x100034a0`. Its first three bytes are `61 67 65` (`age`). Change them to `66 6f 6f` (`foo`), leaving the `: %d\r\n` tail untouched, so the line prints `foo: 70`.

Option A — Hex view:

1. Switch to the **Hex** view (`View -> Hex`).
2. Go to `0x100034a0` and change the three bytes `61 67 65` to `66 6f 6f`.
3. Return to the linear view and reanalyze.

Option B — Python console:

```
bv.write(0x100034a0, b"foo")
print(bv.read(0x100034a0, 10)) # -> b'foo: %d\r\n\x00'
```

Keep the replacement exactly three bytes. If you use a shorter string you must pad it, or `%d` shifts and `printf` reads the wrong argument. A longer string would overwrite the `: %d` tail.

Step 19: Export the patched `.bin`

```
data = bv.read(bv.start, bv.length)
with open("0x0005_intro-to-variables-h.bin", "wb") as f:
    f.write(data)
print(len(data)) # -> 15292
```

A different size means you exported a partial view.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x0005_intro-to-variables-h.bin \
--base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ../uf2conv.py 0x0005_intro-to-variables-h.bin ^
--base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Step 21: Flash and verify `age: 70`

Hold **BOOTSEL**, plug in the Pico 2, and drag `hacked.uf2` onto the `RP2350` drive. Open the serial monitor:

```
age: 70
age: 70
age: 70
...
```

43 became 70, permanently, with one byte changed and no source code.

Part 5: Dynamic — Break at `main` and Hack Live (Project 2)

Step 22: Reflash Project 2 and reload Binary Ninja

Part 4 left the Pico running the patched Project 1 image. Put the original Project 2 back and start a fresh session.

1. Stop any running debug server so the flash script can use the probe:

```
pkill -TERM -f openocd
```

2. Flash the original Project 2 image:

```
./flash.sh 0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.bin
```

3. Start the debug server again (Step 10) and wait for `Listening on port 3333`.
4. Open `0x0008_uninitialized-variables/build/0x0008_uninitialized-variables.bin` with options `(thumb2, thumb2, 0x10000000)` and save a `.bn` file.
5. Connect Binary Ninja again (Step 11): adapter **GDB RSP**, IP `127.0.0.1`, port `3333`.

Confirm the Pico prints `age: 0` and blinks the red LED.

Step 23: Break at `main`

`main` is at `0x10000234` in this project too. The GUI cannot set breakpoints here (Step 13), and you should not use `reset run` from the port while Binary Ninja is attached (it desyncs Binary Ninja's view — Step 13). Use `BP_ADDR`, which arms `main` before Binary Ninja connects:

1. Stop the server (Ctrl-C), then start it parked at `main`:

```
BP_ADDR=0x10000234 ./debug-server.sh
```

2. Connect Binary Ninja (Step 11): adapter **GDB RSP**, IP `127.0.0.1`, port `3333`.

The target is already halted at `main` when Binary Ninja connects, and the sidebar reads `Stopped at 0x10000234`.

If you instead want to reach `main` on an already-connected session, you must Detach, send `reset run` from the port, then reconnect. Arming `main` and resetting while attached leaves the sidebar showing a stale address.

`main` sets up GPIO 16 and then loops: print `age`, turn the LED on, sleep, turn it off, sleep. The whole loop is one function because `blink_and_print` was inlined:

```

10000234 <main>:
10000234: b538      push    {r3, r4, r5, lr}
10000236: f002 ff49  bl     0x100030cc      ; stdio_init_all
1000023a: 2010      movs    r0, #16         ; LED_PIN
1000023c: f000 f83a  bl     0x100002b4      ; gpio_init
10000240: f04f 0501  mov.w   r5, #1
10000244: 2310      movs    r3, #16         ; LED_PIN
10000246: ec45 3044  mcrr    0, 4, r3, r5, cr4 ; gpio_set_dir(16, OUT)
1000024a: 2100      movs    r1, #0          ; age
1000024c: 4809      ldr     r0, [pc, #36]   ; @ 0x10000274 -> "age: %d\r\n"
1000024e: f003 f805  bl     0x1000325c      ; __wrap_printf
10000252: 2410      movs    r4, #16         ; LED_PIN
10000254: ec45 4040  mcrr    0, 4, r4, r5, cr0 ; gpio_put(16, 1)
10000258: f44f 70fa  mov.w   r0, #500
1000025c: f000 fd58  bl     0x10000d10      ; sleep_ms
10000260: f04f 0300  mov.w   r3, #0
10000264: ec43 4040  mcrr    0, 4, r4, r3, cr0 ; gpio_put(16, 0)
10000268: f44f 70fa  mov.w   r0, #500
1000026c: f000 fd50  bl     0x10000d10      ; sleep_ms
10000270: e7eb      b.n     0x1000024a
10000272: bf00      nop
10000274: 10003618  .word   0x10003618

```

Look at the **Registers** widget at `0x1000024e`: `r1` is `0`, which is why the Pico prints `age: 0`.

Step 24: Inspect the GPIO registers live

The GPIO hardware is memory-mapped. Go to each address and watch it change as you step:

Address	Block	Role
<code>0x40028000</code>	<code>IO_BANK0</code>	pin function select and status
<code>0x40038000</code>	<code>PADS_BANK0</code>	pad configuration
<code>0xd0000000</code>	<code>SIO</code>	single-cycle GPIO block driven by <code>mcrr</code>

Step Over through `0x10000254` (`mcrr 0, 4, r4, r5, cr0`) and watch the SIO output register change: this is `gpio_put(16, 1)` turning the red LED on at the hardware level.

Step 25: HACK IT LIVE — change the printed value

1. Press **G**, go to `0x1000024e` (the `bl __wrap_printf`).
2. In the terminal, connect to the OpenOCD command port if you are not already there: `nc 127.0.0.1 4444`. Then at the `OpenOCD>` prompt type `bp 0x1000024e 2 hw` and press Enter. Note `0x1000024e` — Project 2's loop sits at a different address than Project 1's.
3. Click **Resume** in Binary Ninja. The target is already looping, so the comparator fires on the next pass. Binary Ninja stops with `r1 = 0`.
4. In the **Registers** widget, double-click `r1`, type `42`, and press Enter (`0x42 = 66`). The value turns orange.
5. **Move the breakpoint past the call, then Resume.** At the `OpenOCD>` prompt type:

```
rbp 0x1000024e
bp 0x10000252 2 hw
```

`0x10000252` is the instruction right after the `bl __wrap_printf`. Then click **Resume**. The core runs `printf` with `r1 = 0x42` and stops at `0x10000252`. (Not **Step Over** — it steps into the call on this symbol-less `.bin`, and a breakpoint left on the current PC re-traps the step; Step 13 explains both.)

6. Look at your serial monitor and the **Target** tab:

```
age: 66
```

Press **Resume** and the next iteration prints `age: 0` again, because the loop reloads `movs r1, #0` each pass. The live hack is temporary; the static patch makes it permanent.

Step 25b: HACK THE STRING LIVE — change age: to foo:

Same idea as Project 1, different addresses. Here the format string is at `0x10003618` and the `printf` call is at `0x1000024e`.

1. Hit the breakpoint at `0x1000024e` as in Step 25. At the stop, `r0 = 0x10003618` and `r1 = 0`.
2. Write the replacement string to free RAM at `0x20080000` from the command port:

```
mww 0x20080000 0x3a6f6f66
mww 0x20080004 0x0d642520
mww 0x20080008 0x0000000a
```

Bytes `66 6f 6f 3a 20 25 64 0d 0a 00` = `"foo: %d\r\n\0"`.

3. In the **Registers** widget, set `r0` to `0x20080000`.
4. Move the breakpoint past the call and Resume:

```
rbp 0x1000024e
bp 0x10000252 2 hw
```

This iteration prints:

```
foo: 0
```

then stops at `0x10000252`. One iteration only — the loop reloads `r0` each pass. The permanent version is the static patch in Step 28b.

Part 6: Static — Resolve the Functions and Patch (Project 2)

Step 26: Resolve the functions in the Binary Ninja GUI

Use the Project 2 ELF symbol map from Step 4. For each row, press **G** (address), **N** (name), and **Y** (signature):

Same idea as Project 1: **our code plus what it calls**, not the whole SDK. The call chain here is one function longer because `main` also drives the GPIO and sleeps:

```
main
├── stdio_init_all
│   ├── stdio_uart_init — gpio_set_function, uart_init
│   ├── stdio_set_driver_enabled
│   ├── stdio_out_chars_crlf
│   ├── stdio_put_string — strlen
│   └── time_us_64
├── gpio_init
├── __wrap_printf — __wrap_vprintf
└── sleep_ms
```

Two things in this project have **no symbol of their own**, because the compiler inlined them into `main`:

- `blink_and_print` — the `static` helper in our own source is inlined, so there is no `blink_and_print` address to rename. You see its body directly inside `main`.
- `gpio_set_dir` and `gpio_put` — these are `static inline` in the SDK headers, so they compile to the `mccr/SIO` writes you see in `main` rather than to calls.

Project 2 — resolve every function in that chain:

Address	Rename to (N)	Signature (Y)
0x1000015c	_reset_handler	—
0x10000186	platform_entry	void platform_entry(void)
0x1000019a	data_cpy	void data_cpy(void*, void*, void*)
0x100001e4	_init	void _init(void)
0x10000210	frame_dummy	void frame_dummy(void)
0x10000234	main	int main(void)
0x100002b4	gpio_init	void gpio_init(uint)
0x10000278	gpio_set_function	void gpio_set_function(uint, gpio_function_t)
0x10000d10	sleep_ms	void sleep_ms(uint32_t)
0x10002e74	exit	void exit(int)
0x10002e7c	runtime_init	void runtime_init(void)
0x100030cc	stdio_init_all	bool stdio_init_all(void)
0x10003418	stdio_uart_init	void stdio_uart_init(void)
0x100030a4	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x10002ea8	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x10002fb8	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x1000325c	__wrap_printf	int __wrap_printf(const char*, ...)
0x10003198	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x10000f88	uart_init	uint uart_init(uart_inst_t*, uint)
0x10000ef4	time_us_64	uint64_t time_us_64(void)
0x10003558	strlen	size_t strlen(const char*)

Python console shortcut:

```

from binaryninja import Symbol, SymbolType
symbols = {
    0x1000015c: "_reset_handler",      0x10000186: "platform_entry",
    0x1000019a: "data_cpy",           0x100001e4: "_init",
    0x10000210: "frame_dummy",        0x10000234: "main",
    0x100002b4: "gpio_init",          0x10000278: "gpio_set_function",
    0x10000d10: "sleep_ms",           0x10002e74: "exit",
    0x10002e7c: "runtime_init",       0x100030cc: "stdio_init_all",
    0x10003418: "stdio_uart_init",     0x100030a4: "stdio_set_driver_enabled",
    0x10002ea8: "stdio_out_chars_crlf", 0x10002fb8: "stdio_put_string",
    0x1000325c: "__wrap_printf",       0x10003198: "__wrap_vprintf",
    0x10000f88: "uart_init",          0x10000ef4: "time_us_64",
    0x10003558: "strlen",
}
for addr, name in symbols.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))

```

The decompiler now shows `main` initializing GPIO 16 and looping. We make two changes:

- **Move the LED from GPIO 16 to GPIO 17** by patching three `0x10` immediates.
- **Change the printed value from 0 to 66** by patching one `0x00` immediate.

Step 27: Patch 1 — move the LED from GPIO 16 to GPIO 17

GPIO 16 is the red LED; GPIO 17 is the green LED. The pin number appears in three instructions. Change the low byte of each from `10` to `11`:

Address	Instruction	Bytes before	Bytes after	Role
0x1000023a	<code>movs r0, #16</code>	10 20	11 20	pin passed to <code>gpio_init</code>
0x10000244	<code>movs r3, #16</code>	10 23	11 23	pin used by <code>gpio_set_dir</code>
0x10000252	<code>movs r4, #16</code>	10 24	11 24	pin used by <code>gpio_put</code> in the blink loop

In the **Hex** view (lock off), change each byte and reanalyze. Or in the Python console:

```

for addr in (0x1000023a, 0x10000244, 0x10000252):
    bv.write(addr, b"\x11")

```

All three are required. If you patch only the `gpio_set_dir` site, pin 17's output driver is enabled but `gpio_put` still drives pin 16, whose driver was never enabled. Nothing lights up. This is the most common mistake in this lesson.

Step 28: Patch 2 — change the printed value from 0 to 66

`main` loads `age = 0` with `movs r1, #0` at `0x1000024a`. Change the immediate byte from `00` to `42` (`0x42 = 66`):

```

bv.write(0x1000024a, b"\x42")

```

Verify all four patches:

```
for addr in (0x1000023a, 0x10000244, 0x10000252, 0x1000024a):
    print(hex(addr), hex(bv.read(addr, 1)[0]))
# -> 0x1000023a 0x11
# -> 0x10000244 0x11
# -> 0x10000252 0x11
# -> 0x1000024a 0x42
```

Step 28b: Patch the string age: to foo:

The format string starts at `0x10003618` ; change its first three bytes `61 67 65 ( age )` to `66 6f 6f ( foo )`:

```
bv.write(0x10003618, b"foo")
print(bv.read(0x10003618, 10)) # -> b'foo: %d\r\n\x00'
```

Exactly three bytes, same rule as Project 1: a shorter string must be padded, a longer one overwrites the `: %d` tail.

Step 29: Export, convert, and flash

```
data = bv.read(bv.start, bv.length)
with open("0x0008_uninitialized-variables-h.bin", "wb") as f:
    f.write(data)
print(len(data)) # -> 15668
```

```
python3 ../uf2conv.py 0x0008_uninitialized-variables-h.bin \
--base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Hold **BOOTSEL**, plug in the Pico 2, drag `hacked.uf2` onto the `RP2350` drive.

Step 30: Verify

Open the serial monitor:

```
age: 66
age: 66
age: 66
...
```

The **green LED on GPIO 17** now blinks instead of the red one.

We changed the printed value and moved the LED, with four bytes and no source code.

Cheatsheet

Binary Ninja GUI actions

Action	How
Go to address	G
Rename function/symbol	N

Action	How
Set type or signature	<code>Y</code>
Add comment	<code>;</code>
Open Hex view	<code>View -> Hex</code>
Enable hex editing	Toggle the lock in the status bar
Reanalyze after a patch	Right-click function -> <code>Reanalyze</code>
Edit a register live	Double-click the value in the Registers widget, type hex, Enter
Set a breakpoint	Command port only — <code>bp <addr> 2 hw</code> . The GUI cannot set breakpoints (Step 13).
Move a breakpoint	<code>rbp <old addr></code> then <code>bp <new addr> 2 hw</code> . <code>rbp</code> takes an address only.
Confirm what is armed	<code>mdw 0xE0002000 8</code> — each armed breakpoint shows as <code><addr 1></code>
Apply the ELF symbol map	Paste the Python snippet from Step 16 / 26 into the Python Console

OpenOCD server and reset

The server runs with `gdb_breakpoint_override hard` so that flash-writes are never attempted, but that flag is **not** what makes breakpoints work — Binary Ninja's own breakpoints are rejected on length grounds before this setting matters (Step 13). Breakpoints in this lab are armed over the command port with `bp <addr> 2 hw`, which bypasses the GUI entirely.

Action	Command
Connect to the OpenOCD prompt	<code>nc 127.0.0.1 4444</code> (or <code>telnet 127.0.0.1 4444</code>)
Reset and run	<code>reset run</code>
Check core state	<code>targets</code>
Add a breakpoint without the GUI	<code>bp <addr> 2 hw</code>
Remove one breakpoint	<code>rbp <addr></code> — address only, no length, no hw
Remove every breakpoint	<code>rbp all</code>
Start the server parked at <code>main</code>	<code>BP_ADDR=0x10000234 ./debug-server.sh</code> (one-shot; <code>\$env:BP_ADDR</code> on Windows)
Start the server parked in the loop	<code>BP_ADDR=0x1000023e ./debug-server.sh</code> — <code>0x1000024e</code> for Project 2
Break on the loop in a running target	connect first, then <code>rbp <old></code> , <code>bp 0x1000023e 2 hw</code> (or <code>0x1000024e</code> on Project 2), then Resume — repeatable
Make Binary Ninja stepping work	<code>rp2350.dap.core0 configure -rtos none</code> (already in the scripts)
Step without re-trapping	<code>rbp <addr></code> first, then Step Into/Step Over

Action	Command
Reset without desyncing Binary Ninja	Detach , <code>reset run</code> on the port, reconnect — never <code>reset run</code> while attached

Every address and byte we changed

Project	Address	Before	After	Effect
0x0005	0x1000023a	2b	46	prints <code>age: 70</code>
0x0008	0x1000023a	10	11	<code>gpio_init</code> configures GPIO 17
0x0008	0x10000244	10	11	<code>gpio_set_dir</code> enables GPIO 17
0x0008	0x10000252	10	11	<code>gpio_put</code> drives GPIO 17
0x0008	0x1000024a	00	42	prints <code>age: 66</code>
0x0005	0x100034a0	61 67 65	66 6f 6f	string prints <code>foo:</code> instead of <code>age:</code>
0x0008	0x10003618	61 67 65	66 6f 6f	string prints <code>foo:</code> instead of <code>age:</code>

Raw image facts

Item	Value
Build type	Release
Load base address	0x10000000
Project 1 size	15292 bytes
Project 2 size	15668 bytes
Fixed <code>main</code> anchor (both projects)	0x1000018c (reset handler middle <code>blx</code>)
<code>main</code> (both projects)	0x10000234
<code>printf</code> call, Project 1	0x1000023e
<code>printf</code> call, Project 2	0x1000024e
RP2350 UF2 family ID	0xe48bff59

Troubleshooting

Binary Ninja hangs or crashes when you connect (macOS 27)

On macOS 27 with Binary Ninja 6.0.10601, the **GDB MI** and **LLDB** adapters abort inside the debugger core or hang forever at 0% on "The debugger is connecting to the target and preparing the debugger binary view." The crash signature is a macOS crash report for `binaryninja` with `EXC_CRASH (SIGABRT)` and a stack ending in `libdebuggercore.dylib -> std::terminate() -> abort()`. For the hang, `lsof` shows that GDB *did* connect to OpenOCD (`.../arm-none-eabi-gdb --interpreter=mi2` to `127.0.0.1:3333`, `ESTABLISHED`) and the target halted,

yet Binary Ninja never progresses; `sample <pid>` shows it blocked in `libdebuggercore.dylib / libdebuggerui.dylib`. A related issue, [Vector35/debugger #1098](https://github.com/Vector35/debugger/issues/1098), was the bundled **LLDB** crashing on macOS 27 and was fixed before 6.0.

Use the GDB RSP adapter. It is a different code path and it connects cleanly on this setup, so Step 11 already assumes it. If it ever fails, fall back to `arm-none-eabi-gdb` against the same server — the addresses and register values are identical to the GUI steps — and report the GUI failure at <https://github.com/Vector35/debugger/issues>.

If Binary Ninja hangs, you must force-quit it; the connect dialog has no working Cancel. The static steps (resolve, patch, export, flash) never touch the debugger and always work.

The GUI refuses to set a breakpoint

This is a **Binary Ninja bug, not a misconfiguration**, and it is expected on BN 6.0.10601. Binary Ninja sends `Z0,<addr>,1`; the Cortex-M33 comparators require 2 bytes, so OpenOCD answers `only breakpoints of two bytes length supported`. It affects every address, both `Toggle Breakpoint` and `Add Hardware Breakpoint`, and the dialog's **Size** field is disabled. Set `gdb_breakpoint_override` either way — no effect.

Work around it by arming the breakpoint from the OpenOCD command port **after** Binary Ninja is connected. Full procedure in Step 13.

GDB MI hangs when you connect (a breakpoint already existed)

With the **GDB MI** adapter, if Binary Ninja already has a breakpoint set when you connect, the session **hangs**. This is a Binary Ninja bug. The working order is:

1. Start the server parked, e.g. `BP_ADDR=0x10000234 ./debug-server.sh`.
2. Connect with the **GDB MI** adapter.
3. Only *then* set hardware breakpoints in the UI.

Never have a breakpoint in the binary view before the GDB MI connection. If it hangs, quit Binary Ninja, restart the server with `BP_ADDR`, and connect again before adding any breakpoints.

Step Into / Step Over does nothing (PC never moves)

Two independent causes, both fixed. Full explanation in Step 13.

1. **hwthread RTOS makes OpenOCD fake the step.** Binary Ninja sends `vCont;s` with thread id 0; the RP2350 config's `-rtos hwthread` makes the current thread id 1, so OpenOCD logs `fake step thread 0` and replies with a stop without stepping. Fix: `rp2350.dap.core0 configure -rtos none`. The launcher scripts already pass this.
2. **A breakpoint on the current PC re-traps the step.** OpenOCD's step-over-breakpoint logic fails with `Duplicate Breakpoint address` and the PC stays put. Fix: `rbp <addr>` before stepping.

To tell them apart, turn on OpenOCD logging (`log_output /tmp/ocd.log` then `debug_level 3` on the command port) and look for `fake step` versus `Duplicate Breakpoint`.

```
zsh: bad CPU type in executable: cmake
```

An Intel `x86_64` tool is on your `PATH` on Apple Silicon. Run Step 2: `export PATH="/opt/homebrew/bin:$PATH"`, then `hash -r`. Add it to `~/.zshrc` to make it permanent.

My addresses do not match this guide

You probably built `Debug`. This lesson is a `Release` build. Re-run Step 3 with `-DCMAKE_BUILD_TYPE=Release`. A `Debug` build moves the SDK functions and keeps `blink_and_print` separate, so Project 2's `main` is not at `0x10000234`.

A breakpoint never fires

First, confirm you actually armed one. The GUI cannot set breakpoints on this target (Step 13): Binary Ninja sends `Z0,<addr>,1` and OpenOCD rejects the 1-byte length, so `Debugger -> Toggle Breakpoint (F2)` and `Debugger -> Add Hardware Breakpoint...` (F3) both fail with `only breakpoints of two bytes length supported` and nothing lands in the **Breakpoints** widget. Arm it on the command port instead.

Then check the order and the state:

- **Arm it only after Binary Ninja is connected.** OpenOCD flushes every breakpoint when a client attaches (`breakpoint_remove_all_internal -> Delete all breakpoints`), so anything armed earlier is gone. This also applies to `BP_ADDR` on the startup command line, and to `hbreak` followed by `detach` in GDB.
- **Verify it is armed:** `mdw 0xE0002000 8`. You should see your address with the low bit set (`0x10000234 -> 0x10000235`). All zeros means nothing is armed — re-read this first, because it distinguishes "not armed" from "armed but never reached".
- **Is the core running?** `poll` on the command port should not report a halt. If it is stopped, click **Resume**.
- **Does the address get reached again?** `main` runs once per reset, so use `BP_ADDR` at startup (Step 10) rather than `reset run` while attached. Loop addresses such as `0x1000023e` fire on the next pass with no reset — arm them and click **Resume** in Binary Ninja.
- **Binary Ninja reports these stops as** `SingleStep`, **not** `Breakpoint`, and leaves the **Breakpoints** widget empty. That is expected — the core really is halted on a comparator Binary Ninja knows nothing about.

It worked for a second, then stopped (Binary Ninja's view desyncs)

This is the most common failure, and it has one main cause: **driving the core from the OpenOCD command port while Binary Ninja is connected.**

- If you send `reset run` from the port while attached, the core resets, runs, and halts at your breakpoint — but Binary Ninja never receives the stop event. Its sidebar keeps showing the *previous* location, so **Step** and **Resume** act on a stale PC and appear to do nothing. Verified: target at `0x1000023e` while the sidebar still read `Stopped (SingleStep) at 0x10003020`.
- If the OpenOCD process dies (or you restart it) while attached, Binary Ninja keeps believing it is connected: the sidebar stays, but the menu shows **Pause** enabled and **Resume/Step** disabled because Binary Ninja last saw the target *running*.

Recovery: **Detach, then reconnect.** If Detach does nothing (the connection is already dead), restart Binary Ninja — its menu still shows a session that no longer exists.

Prevention:

- Stop at `main` with `BP_ADDR` on a fresh server start, not with `reset run` while attached.
- For loop addresses, arm the comparator and click **Resume** in Binary Ninja. Let Binary Ninja be the thing that starts the core.
- If you must reset, **Detach first**, `reset run`, then reconnect.

- Never leave a breakpoint on the PC you are about to step or resume from (see the stepping section above).

If the stop is at `0x1000320c` rather than your breakpoint, you stopped inside `stdio_uart_out_flush`, not at `main`. See the next section.

The target "blows past" `main` and stops at `0x1000320c` instead

`0x1000320c` is inside `stdio_uart_out_flush`:

```
1000320c: 6993    ldr r3, [r2, #24]
1000320e: 071b    lsls r3, r3, #28
10003210: d4fc    bmi.n 0x1000320c
```

That is the UART transmit-FIFO drain loop inside `printf`, so the core is running `main`'s loop and simply spends nearly all its time there. The breakpoint at `main` did not fire because `main`'s entry (`0x10000234`) runs exactly **once per reset**. If you arm the breakpoint after the reset, or set it while the target is already running and just resume, the core is already past `0x10000234` and will never re-execute it. Either arm the breakpoint **before** resetting, or break inside the loop at `0x1000023e`, which fires every iteration.

`0x1000320c` **is not a function**. It is one instruction inside `stdio_uart_out_flush`, which starts at `0x10003208`:

```
10003208 <stdio_uart_out_flush>:
10003208: 4b02    ldr r3, [pc, #8] ; @ 0x10003214
1000320a: 681a    ldr r2, [r3]
1000320c: 6993    ldr r3, [r2, #24] ; the core sits here while the UART drains
1000320e: 071b    lsls r3, r3, #28
10003210: d4fc    bmi.n 0x1000320c
10003212: 4770    bx lr
10003214: 20000850 .word 0x20000850
```

If Binary Ninja has created a function at `0x1000320c` (for example because the debugger stopped at that PC), the decompiler shows garbage: registers named `entry_r4` / `entry_r5`, and stores to invented constants like `0x3a` and `0xffffffff6`. Delete that bogus function (right-click it -> `Delete Function`, or put the cursor on it and press `U` to undefine) and reanalyze. The real function is `stdio_uart_out_flush` at `0x10003208`.

The console floods with Failed to read memory at `0xf0000000`

Core1 is exposed. The scripts must run with `USE_CORE=0`. Stop the server, confirm only `core0` is reported, restart, then restart Binary Ninja.

Connect to Remote Process is greyed out and Pause does nothing

Binary Ninja is in a stale session, usually because the debug server restarted while attached. Quit and reopen Binary Ninja (or the `.bndb`) and connect again.

The decompiler still shows the old value after patching

Right-click the function and choose `Reanalyze`.

Project 2's LED does not light at all after patching

You patched only some of the three GPIO 16 sites. All three of `0x1000023a`, `0x10000244`, and `0x10000252` must change.

Fallback: do the dynamic steps with GDB (macOS 27)

If Binary Ninja's debugger crashes on attach on macOS 27 (see Troubleshooting), you can still do the live hack with the ARM GDB from the toolchain, against the same OpenOCD server. The addresses and register values are identical to the GUI steps.

Start the debug server (Step 10), then in a new terminal:

```
arm-none-eabi-gdb
```

At the `(gdb)` prompt:

```
set architecture armv8-m.main
target extended-remote :3333
hbreak *0x1000023e
continue
```

Do **not** run `monitor reset run` before `hbreak`. `0x1000023e` is inside `main`'s loop, so the breakpoint fires on the next iteration with no reset. If you reset first, the core runs `main` and you will not catch it.

GDB stops at the `printf` call. Confirm the value, change it, and let it run:

```
info registers pc r1          # pc = 0x1000023e, r1 = 0x2b
set $r1 = 0x46
stepi
continue
```

The serial monitor prints `age: 70` for the iteration you changed — the same temporary live hack as editing `r1` in the Binary Ninja Registers widget. When you are done, press `Ctrl-C`, then `detach` and `quit`.

If you specifically want to stop at `main` (`0x10000234`), remember its entry runs only once per reset, so the breakpoint must be armed *before* the reset:

```
monitor reset halt
hbreak *0x10000234
continue
```

If you instead set it while the target is running and just `continue`, you will "blow past" `main` and catch the core inside `printf` — in this build at `0x1000320c`, the `stdio_uart_out_flush` UART-drain loop.

Project 2 is the same with the other call site and value:

```
hbreak *0x1000024e
continue
info registers pc r1          # pc = 0x1000024e, r1 = 0
set $r1 = 0x42
stepi
```

`hbreak` sets a hardware breakpoint, which is required for read-only flash. It works from plain GDB because GDB sends the 2-byte length the Cortex-M33 comparators need — the same length Binary Ninja gets wrong, which is why the GUI cannot set breakpoints at all here (Step 13).

Glossary

Term	Definition
<code>.bss</code>	Section for uninitialized global variables; zeroed by startup code
<code>.data</code>	Section for initialized global variables; copied from flash to SRAM at boot
<code>.elf</code>	Linked image with the symbol table; the ground truth for addresses and names
<code>.rodata</code>	Read-only section for constants and string literals; stays in flash
GPIO	General Purpose Input/Output — controllable pins on the microcontroller
Hardware breakpoint	A breakpoint serviced by the CPU comparators, required for read-only flash
Inlining	The optimizer replacing a function call with the function body; why <code>blink_and_print</code> disappears in <code>Release</code>
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative <code>ldr</code>
MMIO	Memory-mapped I/O — hardware registers accessed as memory addresses
SIO	Single-cycle I/O — the fast GPIO block in the RP2350, at <code>0xd0000000</code>
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Prove the behavior dynamically, resolve the names from the ELF, then patch the bytes and flash.